ISA/IEC 62443 SL-3 Checklist Generator

User Manual — build product-specific 3-3 / 4-2 SL-3 testing cheat sheets

A single-file, zero-dependency companion to the ICS/OT/IoT Threat Modeler. Describe a component, add manuals / datasheets / URLs, pick any AI model, and generate a detailed, field-ready SL-3 testing cheat sheet PDF — mapped to every ISA/IEC 62443-3-3 system requirement and 4-2 component requirement, with step-by-step native and external-tool test procedures.

Version 1.0.0 | Python 3.9+ | No third-party packages required

1. What this tool is

The **62443 SL-3 Checklist Generator** is a standalone Python program that runs a small local web app in your browser. You describe an ICS/OT/IoT product, optionally attach its manuals, datasheets, and web links, choose any AI model (with your own API key), and the tool produces a **product-specific ISA/IEC 62443-3-3 / 4-2 · SL-3 Testing Cheat Sheet** as a print-ready landscape PDF. It is the checklist companion to the ICS/OT/IoT Threat Modeler and follows the same design: one self-contained file, a model-agnostic provider router, and the same four-step workflow.

What it produces

For the confirmed component type, the sheet covers every Foundational Requirement (FR1–FR7) with the applicable system requirements (SR), component requirements (CR), and the component-specific EDR/HDR/NDR/SAR rows. Each row gives numbered native configuration/verification steps, the specific external tools to validate that control (with exact commands or click-paths), the pass criteria / evidence to capture, and product/firmware notes — plus an **External Test Tools** table.

[sample]

2. Requirements & installation

Zero dependencies

The tool uses the Python standard library only — there is nothing to pip install. The landscape PDF is produced by a built-in PDF engine; PDF/DOCX text and images are parsed with the standard library; web pages and manuals are fetched with urllib.

- **Python 3.9 or newer** (Windows, macOS, or Linux).
- A network connection to your chosen AI provider (and any URLs you scrape).
- An API key for at least one AI provider — or a local/OpenAI-compatible endpoint (Ollama, LM Studio, vLLM), which needs no key.
- **No second key.** The selected model's key does everything, including web search — there is no separate search-API key to obtain.

Run it

```
python3 ics_62443_sl3_checklist_generator.py
```

Then open the printed address (default http://127.0.0.1:8010) in your browser. Useful flags:

```
python3 ics_62443_sl3_checklist_generator.py --port 9000 --open python3
ics_62443_sl3_checklist_generator.py --stats ANTHROPIC_API_KEY=sk-ant-... python3
ics_62443_sl3_checklist_generator.py
```

- **--open** — open the UI in your browser automatically.
- **--port N** — serve on a specific port (auto-tries the next few if busy).
- **--db PATH** — where saved checklists are stored (SQLite).
- **--stats** — print the embedded 62443 catalog counts and exit.

Your API keys are stored only in your browser and are sent to the tool solely to call the provider you selected. Nothing is sent anywhere else.

3. The four-step workflow

Step 1 — Choose your AI model & key

Pick the provider and model that will write the checklist and paste that provider's API key. You can type a custom model id or point at a local endpoint. That single key does everything — including web search — so there is no second key to enter. Click **Verify & continue** and the tool validates the key.

Provider	Notes	Web search
Anthropic (Claude)	console.anthropic.com — vision-capable	Yes
OpenAI	platform.openai.com — vision-capable	Yes
Google Gemini	aistudio.google.com — vision-capable	Yes
DeepSeek	platform.deepseek.com — OpenAI-compatible	No*
Kimi (Moonshot)	OpenAI-compatible	No*
Local / OpenAI-compatible	Ollama, LM Studio, vLLM — set Base URL, no key	No*

Web search runs natively through the model's own key (Anthropic web search, Google Search grounding, OpenAI web search). *Providers without a native search tool still work — they use the model's built-in knowledge plus any URLs and files you add.

Step 2 — Describe the component

Name the product as specifically as you can — vendor, model, family, firmware/version (e.g. "Siemens SIPROTEC 5 7SJ85 overcurrent/feeder protection relay, DIGSI 5"). Add scope/context: deployment, target zone, and what the assessor can or cannot touch.

Step 3 — Add sources

- **AI web search** — tick "Let the AI search the web on its own" (on by default). The model searches the web itself, using your model key, for the product's manual, datasheet, and security guides — no separate search key.
- **Source URLs** — optionally paste product pages, online manuals, or datasheet PDFs (one per line), fetched and scraped live.
- **Uploads** — manuals and datasheets (PDF, DOCX, TXT, images). Very large manuals (400+ pages) are split into sections and fully read.

Step 4 — Confirm the type & generate

The AI identifies the product and its 62443-4-2 component type. You **confirm or override** the type before generation — it drives which component-specific rows appear:

Code	Type	Typical products
EDR	Embedded Device	PLC/PAC, RTU, IED/relay, drive/VFD, meter, field instrument
HDR	Host Device	Windows/Linux server or workstation, OS-based appliance
NDR	Network Device	Switch, router, firewall, gateway, protocol converter, AP
SAR	Software Application	SCADA/HMI, historian, OPC server, engineering software

A product can be more than one type (e.g. a gateway that is EDR + NDR) — tick all that apply. The tool then generates the full sheet, streaming progress as it reads sources and maps each requirement.

4. Reading & exporting the cheat sheet

Each Foundational Requirement (FR1–FR7) is a section; each row has five columns:

- **Req (3-3 / 4-2)** — the requirement id (e.g. SR/CR 1.1, EDR 3.11).
- **Requirement** — the control name.
- **Where & how to verify** — numbered native steps, then **Ext. tools:** with the specific tools and exact commands/click-paths.
- **Pass criteria / evidence** — what a pass looks like and what to capture.
- **Notes / firmware** — version caveats, compensating-control and N/A markers.

Export options

- **Download PDF** — the landscape SL-3 cheat sheet matching the sample product sheets.
- **Printable HTML** — the same layout as an HTML page you can print to PDF.
- **Refine** — type a natural-language change ("add IEC 61850 GOOSE/MMS steps to FR3", "treat it as EDR+NDR") and the sheet regenerates in place.
- **Saved** — every sheet is stored locally; reopen, re-export, or delete from the Saved panel.

5. How the tool chooses tools & writes steps

The value of the sheet is matching the *right* validation method to each requirement for the *specific* product — not repeating one or two tools everywhere. The tool is instructed to:

- Use the vendor's own native tools/consoles/CLIs/config files/logs first;
- Then choose appropriate **open-source** and **commercial** tools per requirement — e.g. openssl/testssl.sh for certificates; the correct protocol client (pycomm3, python-snap7, pymodbus, OPC UA, IEC 61850, DNP3) for auth/read tests; Hydra/Medusa for lockout (isolated); fuzzers for input validation (isolated); CIS-CAT/OpenSCAP/Lynis for hardening baselines; passive OT monitors (Claroty, Nozomi, Dragos, Tenable.ot) for monitoring;
- Include your **ReadOnlyEnum.py** for read-only host/interface enumeration where it applies;
- **Research** the product documentation and the web for additional, product-specific tooling — the built-in palette is a starting point, not a limit;
- Write **numbered, step-by-step** procedures for both the native path and each external tool, with real menu paths, settings, ports, protocols, commands, and flags.

Richer sources (a real manual, a hardening guide, the vendor security advisory) produce a sharper, more accurate sheet.

6. Requirement coverage

The embedded ISA/IEC 62443 catalog drives complete coverage. The model produces a row for every applicable requirement, using N/A justifications where a control does not apply.

Foundational Requirement	Scope
FR1 — Identification & Authentication	SR/CR 1.1–1.13 (+ CR 1.14, NDR 1.6/1.13)
FR2 — Use Control	SR/CR 2.1–2.12 (+ CR/EDR 2.13, component mobile-code rows)
FR3 — System Integrity	SR/CR 3.1–3.9 (+ CR/EDR/HDR/NDR 3.10–3.14)
FR4 — Data Confidentiality	SR/CR 4.1–4.3
FR5 — Restricted Data Flow	SR/CR 5.1–5.4 (+ NDR 5.2/5.3)
FR6 — Timely Response to Events	SR/CR 6.1–6.2
FR7 — Resource Availability	SR/CR 7.1–7.8

Component-specific rows (EDR/HDR/NDR/SAR) are added automatically from the type you confirm. The tool embeds 51 system requirements, 54 component requirements, and the EDR/HDR/NDR/SAR sets — no internet needed for the standard itself.

7. Best results

- Give a precise product name and firmware/version — it sharpens native menu paths and version notes.
- Attach the real user manual and any hardening/security guide; add the vendor security-advisory URL if you have it.
- For 400+ page manuals, a URL to the online manual can be faster than uploading; the whole document is read either way.
- Pick a strong, current model for the deepest step-by-step detail; vision models can also read diagram images.
- Use **Refine** to iterate — add protocol-specific tests, expand the tools table, or re-scope the component type.
- Confirm the component type carefully; it determines which EDR/HDR/NDR/SAR rows appear.

8. Safety, authorization & scope

The sheets reference active test tooling (fuzzers, brute-force, exploit validation), marked for **isolated / representative environments with written authorization only**. Confirm exact menu paths and commands against the deployed firmware and your project hardening baseline before testing, and never run intrusive tools against production control systems. The tool produces assessment guidance; it does not connect to or test your devices itself.

9. Troubleshooting

The key was rejected.

Paste the full API key with no extra spaces (Anthropic keys start with sk-ant-), confirm it is active and funded, and that the provider/model match the key. For local endpoints, set the Base URL.

A URL would not scrape.

Some vendor sites block automated fetching. Download the manual/datasheet and upload it, or paste a direct link to the PDF. The tool never bypasses site restrictions.

The port is busy.

The tool automatically tries the next few ports; or start it with --port to choose one.

A row looks generic.

Add more sources (the real manual, a hardening guide, the vendor advisory) and regenerate, or use Refine to ask for deeper native steps or specific protocol tests for that FR.

Can I run it fully offline?

Yes, with a local model (Ollama/LM Studio/vLLM). The 62443 catalog and PDF engine are built in; only web scraping/search and remote AI providers need a connection.

Where are my saved checklists?

In a local SQLite database (default in your home folder; change with --db). Nothing is uploaded to any external service.

10. At a glance

Item	Value
Run	`python3 ics_62443_sl3_checklist_generator.py`
Default URL	`http://127.0.0.1:8010`
Dependencies	`None (Python 3.9+ standard library only)`
AI providers	`Anthropic, OpenAI, Gemini, DeepSeek, Kimi, local/OpenAI-compatible`
Web search	`Built-in — the AI model searches with your key (Anthropic, OpenAI, Google)`
Component types	`EDR, HDR, NDR, SAR (and combinations)`
Output	`Landscape SL-3 cheat-sheet PDF + printable HTML; saved locally`